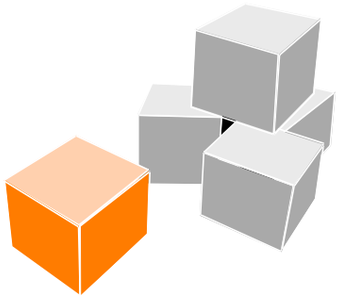
Concepts of Programming Languages

FUNCTIONS (CONT‘D) & RECURSION

Prof. Dr. Mira Mezini



Software Technology Group
TU Darmstadt | FB Informatik

Outline

- **Implementing First-Class Functions**

- **Recursion**
  - Understanding recursion
  - Implementing recursion

- **Reflecting on representational choices**
  - Data structures for environments, numbers, functions
  - Types of interpreters

FLAE: A Language with First-Class Functions

- **Function definitions are expressions in FLAE**
  - They can appear everywhere within other compound expressions
  - Functions are values just like numbers
  - Function values can be passed to and returned by other functions

- Examples of FLAE programs:

```
( (fun (x) (+ x x)) 3 )

(let (inc (fun (x) (+ x 1)) )
    (+ (inc 4) (inc 5)))

(let (x 3)
    (fun (y) (+ x y)) )
```

Concrete and Abstract Syntax of FLAE

```
<F1LAE> ::= <num>
          | (+ <F1LAE> <F1LAE>)
          | (- <F1LAE> <F1LAE>)
          | (let (<id> <F1LAE>) <F1LAE>)
          | <id>
          | (<id> <F1LAE>)
```

How will the concrete and abstract syntax of FLAE differ?

```
sealed trait F1LAE
case class Num(n: Int) extends F1LAE
case class Add(lhs: F1LAE, rhs: F1LAE) extends F1LAE
case class Sub(lhs: F1LAE, rhs: F1LAE) extends F1LAE
case class Let(name: String, namedExpr: F1LAE, body: F1LAE) extends F1LAE
case class Id(name: String) extends F1LAE
case class App(funName: String, arg: F1LAE) extends F1LAE
```

Concrete and Abstract Syntax of FLAE

- Concrete syntax

```
<FLAE> ::= <num>
         | (+ <FLAE> <FLAE>)
         | (- <FLAE> <FLAE>)
         | (let (<id> <FLAE>) <FLAE>)
         | <id>
         | (fun (<id>) <FLAE>)
         | (<FLAE> <FLAE>)
```

fun defs are now expressions

first component of an application is a FLAE not an id as in F1LAE

- Abstract syntax

```
sealed abstract class FLAE
  case class Num(n: Int) extends FLAE
  case class Add(lhs: FLAE, rhs: FLAE) extends FLAE
  case class Sub(lhs: FLAE, rhs: FLAE) extends FLAE
  case class Let(name:String, namedExpr:FLAE, body:FLAE) extends FLAE
  case class Id(name: String) extends FLAE
  case class Fun(param: String, body: FLAE) extends FLAE
  case class App(funExpr: FLAE, arg: FLAE) extends FLAE
```

Interpreting FLAE

1. Implement an interpreter for **FLAE** that employs **substitution**
   – to facilitate the comparison to FLAE and F1LAE
   – to define a „reference“ specification of the semantics

2. Replace substitutions with **environments**

Interpreting FLAE With Substitution

- What does the interpreter produce, i.e., what are the **values** of FLAE?
- How do we represent functions inside the interpreter?
- What needs to be done to turn LAE into FLAE?

FLAE with Substitution

1. Extend the class of values
   - LAE produced Scala integers.
   - The interpretation of FLAE expressions can also produce functions
   - Return values of FLAE are `Num` or `Fun`

2. **Add clauses for function definition expressions** to the substitution and the interpreter functions

3. **Modify substitution** and **interpretation of application expressions**

FLAE with Substitution

Values

```
abstract class Value
case class VNum(n: Int) extends Value
case class VFun(param: String, body: FLAE) extends Value
```

AST

```
...
case class Let(name: String, namedExpr: FLAE, body: FLAE) extends FLAE
case class Id(name: String) extends FLAE
case class Fun(param: String, body: FLAE) extends FLAE
case class App(funExp: FLAE, arg: FLAE) extends FLAE
```

Subst

```
def subst(expr: FLAE, substId: String, value: FLAE): FLAE = expr match {
...
  case Fun(param, body) => ???

  case App(funExpr, argExpr) => ???

}
```

FLAE with Substitution

Values

```
abstract class Value
case class VNum(n: Int) extends Value
case class VFun(param: String, body: FLAE) extends Value
```

AST

```
...
case class Let(name: String, namedExpr: FLAE, body: FLAE) extends FLAE
case class Id(name: String) extends FLAE
case class Fun(param: String, body: FLAE) extends FLAE
case class App(funExp: FLAE, arg: FLAE) extends FLAE
```

Subst

```
def subst(expr: FLAE, substId: String, value: FLAE): FLAE = expr match {
...
  case Fun(param, body) =>
    if (param == substId) ...
    else
      ...

  case App(funExpr, argExpr) =>
      ...
```

FLAE with Substitution

Values

```
abstract class Value
case class VNum(n: Int) extends Value
case class VFun(param: String, body: FLAE) extends Value
```

AST

```
...
case class Let(name: String, namedExpr: FLAE, body: FLAE) extends FLAE
case class Id(name: String) extends FLAE
case class Fun(param: String, body: FLAE) extends FLAE
case class App(funExp: FLAE, arg: FLAE) extends FLAE
```

Subst

```
def subst(expr: FLAE, substId: String, value: FLAE): FLAE = expr match {
...
  case Fun(param, body) =>
    if (param == substId) Fun(param, body)
    else
      Fun(param, subst(body, substId, value))

  case App(funExpr, argExpr) =>
    ...
```

FLAE with Substitution

Values

```
abstract class Value
case class VNum(n: Int) extends Value
case class VFun(param: String, body: FLAE) extends Value
```

AST

```
...
case class Let(name: String, namedExpr: FLAE, body: FLAE) extends FLAE
case class Id(name: String) extends FLAE
case class Fun(param: String, body: FLAE) extends FLAE
case class App(funExp: FLAE, arg: FLAE) extends FLAE
```

Subst

```
def subst(expr: FLAE, substId: String, value: FLAE): FLAE = expr match {
...
  case Fun(param, body) =>
    if (param == substId) Fun(param, body)
    else
      Fun(param, subst(body, substId, value))

  case App(funExpr, argExpr) =>
     App(subst(funExpr, substId, value), subst(argExpr, substId, value))
```

FLAE with Substitution: Interpreter

```
def interp(expr: FLAE): Value = expr match {
  ...

  case Fun(param, body) => ???

  case App(funExpr, argExpr) => ???

  }
```

FLAE with Substitution: Interpreter

```
def interp(expr: FLAE): Value = expr match {
  ...

  case Fun(param, body) => VFun(param, body)

  case App(funExpr, argExpr) => ???

  }
```

FLAE with Substitution: Interpreter

```
def interp(expr: FLAE): Value = expr match {
  ...

  case Fun(param, body) => VFun(param, body)

  case App(funExpr, argExpr) => interp(funExpr) match {
    case VFun(param, body) =>
            interp(subst(body, param, argExpr))
    case v1 => error(s"Expected funct value but got $v1")
    }

  }
```

FLAE with Environments

- Replace substitution with environment lookup
- Preserve static scoping

Question

To avoid dynamic scoping, in F1LAE with environments we used an empty environment for evaluating function applications. Can we apply the same trick here?

Answer: No
Unlike F1LAE, functions in FLAE are not defined in isolation. When they are defined, there may be identifiers used in their body that are statically scoped. They should not be deleted by the fresh environment.

FLAE with Environments

Compare manual evaluations of the following…

```
interp(App("f", Num(4)),
        Map("f" -> FunDef("y", Add("x" "y")) ))
```

```
interp(Let("x", Num(3),
                App( Fun("y", Add("x" "y")) Num(4))) )
```

The fist one has a problem: x inside the function is not defined.
The second one is OK, but would fail with a fresh environment.

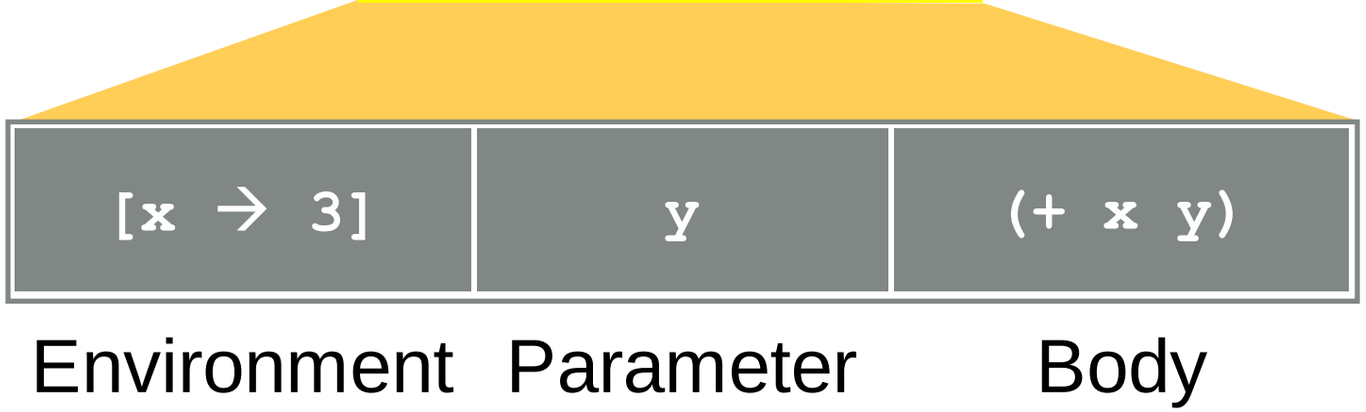
Any idea how to avoid this?

Function Closures

- Remember the current environment at function definition time.
- Use it for binding free identifiers in function bodies at function application time.

- Function + environment = **closure**

```
(let (x 3) (fun (y) (+ x y)) )
```



FLAE with Environments

1. Define a data type for representing FLAE function values (**closures**)

2. Modify the definition of environments to use the new function values

3. Modify the interpreter to:
   - use the environment for deferring substitutions
   - return closures as the result of evaluating function definitions
   - when evaluating function applications: use the environment of the closure returned by evaluating the function sub-expression

Quiz

Do we really need **`let`** expressions in a language with first-class functions?

Quiz

Do we really need **`let`** expressions in a language with first-class functions?

We have two binding constructs, but we only need one.

“Fun” is more powerful than “let” (because of parameters, which can’t be expressed in “let”). We can express let as functions

```
(let (x 4) (+ x 3)) <==> ((fun (x) (+ x 3)) 4)
```

=> Recall “core“ and „desugaring“

“Closures” for Java

Closures are one of the features added in Java 8.

They are objects that implement the Callable interface

```
public interface Callable<V> {
   V call();
}
```

Main driving force: concurrent, parallel programming on multi-cores.

Are Java closures real closures?

Not perfect closures … Local variables referenced from a lambda expression must be **final** or **effectively final***

***A variable or parameter whose value is never changed after it is initialized is effectively final [up to what the compiler can infer]

Outline

- **Implementing First-Class Functions**

- **Recursion**
  - Understanding recursion
  - Implementing recursion

- **Reflecting on representational choices**
  - Data structures for environments, numbers, functions
  - Types of interpreters

Languages With Functions So Far

- We started with **first-order functions (F1LAE)**.
- Function definitions are not expressions in the language - they are passed as a parameter to the interpreter.
- Functions are not values.

```
<F1LAE> ::= <num>
          | (+ <F1LAE> <F1LAE>)
          | (- <F1LAE> <F1LAE>)
          | (let (<id> <F1LAE>) <F1LAE>)
          | <id>
          | (<id> <F1LAE>)
```

```
interp(
  App("f", 10),
  Map(
    "f" -> FunDef("x", App("g", Add("x", 3))),
    "g" -> FunDef("y", Sub("y", 1)))
```

Question: Is special treatment for recursion needed in F1LAE?

No, nothing special needs to be done for recursion.

Just need to add conditionals

Languages With Functions So Far

- Next, we studied/implemented first-class functions (FLAE).
- Function definitions are expressions that evaluate to values.

```
<CFLAE> ::= <num>
          | (+ <CFLAE> <CFLAE>)
          | (- <CFLAE> <CFLAE>)
          | (let {<id> <CFLAE>} <CFLAE>)
          |  <id>
          | (fun (<id>) <CFLAE>)
          | (<CFLAE> <CFLAE>)
          | (* <CFLAE> <CFLAE>)
          | (if0 <CFLAE> <CFLAE> <CFLAE>)
```

Assume we have implemented CFLAE = FLAE extended with ***** and **if0-conditional.**

Question: Is special treatment for recursion needed in CFLAE?

Recursive Factorial in CFLAE

```
(let fact
    (fun (n) ((if0 n) (1) (* n (fact (- n 1))))
    (fact 5))
```

What does the application (fact 5) evaluate to?

With **static scoping**:

- The first recursive call will fail.
- `fact` is bound in the body of `let`, (i.e., the app expr (`fact 5)`) but not in the named expression (i.e., the definition of `fact)`.
- So, we need other semantics for recursive binding constructs.

Is this also the case with dynamic scoping?

RCFAE – A Language with Recursion

Different semantics, different names.

We introduce a new binding construct **`letrec`**

```
<RCFLAE> ::= <num>
           | (+ <RCFLAE> <RCFLAE>)
           | (- <RCFLAE> <RCFLAE>)
           | (let {<id> <RCFLAE>} <RCFLAE>)
           |  <id>
           | (fun (<id>) <RCFLAE>)
           | (<RCFLAE> <RCFLAE>)
           | (* <RCFLAE> <RCFLAE>)
           | (if0 <RCFLAE> <RCFLAE> <RCFLAE>)

           | (letrec (<id> <RCFAE>) <RCFAE>)
```

What's the meaning of **`letrec`**?

It makes the new binding available in both its body and in its named expression.

```
(letrec fact
    (fun (n) ((if0 n) (1) (* n (fact (- n 1))))
    (fact 5))
```

Bindings and Environments

- Binding constructs such as `let` transform environments
- Given a current environment `e` – we call this, **the ambient environment -** the evaluation of `let` produces two new environments, one for each of its sub-expressions.

for named expression

$$\rho_{\texttt{let,named}}(\texttt{e}) = \texttt{e}$$

for body

$$\rho_{\texttt{let,body}}(\texttt{e}) = \texttt{e} + (\texttt{boundId} \rightarrow \texttt{boundValue})$$

Bindings and Environments

How do these transformations need to change for `letrec`?

for named expression

$\rho_{\texttt{letrec,named}}(\texttt{e}) = ???$

for body

$\rho_{\texttt{letrec,body}}(\texttt{e}) = ???$

Unlike with `let` now the **new binding must be made available in both the body and the named expression**.

`letrec`'s Environment for Body

$\rho_{\texttt{letrec,body}}$ of `letrec` remains probably the same …

Created by the evaluation of the named expression part

for body

```
ρ_letrec,body(e) =
e + ("fact" ->
      Closure("n", If0( /*…fact…*/ ), ?) )
```

But what should be put instead of the question mark**?**

`let`'s Environment for Body

```
ρlet,body(e) = e +
    ("fact" -> Closure("n", If0( /*…fact…*/ ), e))
```

`let` closes the closure over the ambient environment `e` in which we interpret `let`.

Obviously, not OK for `fact`

- The ambient environment when we evaluate `let` does not have a binding for `fact` - first recursive call fails
- We need a binding for `fact` in the environment of its closure.

`letrec`'s Environment for the Named Expression

The evaluation of the named expression must ensure that the right environment is computed for the `fact` closure.

```
(letrec (fact (fun (n) (if0 ... fact ...) ) ) (fact 5))
```

$\rho_{\texttt{letrec},\texttt{named}}(\texttt{e})$ = environment for evaluating this func definition expression, i.e., environment to put in the resulting closure

`letrec`'s Environment for the Named Expression

So far, we know that with

$\rho_{\texttt{letrec},\texttt{named}}(\texttt{e}) = \texttt{e}$

no recursive calls are possible.

It should be something like:

$\rho_{\texttt{letrec},\texttt{named}}(\texttt{e}) =$

```
e +
("fact" -> Closure("n", If0( /*…fact…*/ ), ???))
```

letrec's Environment for the Named Expression

Let's try this

$\rho_{\texttt{letrec,named}}$(e) =

```
e +
("fact" -> Closure("n", If0( /*…fact…*/ ), e))
```

```
(letrec (fact (fun (n) (if0 ... fact ...) ) ) (fact 5))
```

```
interp(FunDef("n", If0( /*…fact…*/ )), ρletrec,named(e)) =
  Closure("n",
        If0( /*…fact…*/ ),
        e +
        ("fact" -> Closure("n", If0( /*…fact…*/ ), e))
        )
```

letrec's Environment for the Named Expression

Which further means:

$\rho_{letrec,body}$(e) =

```
e + ("fact" -> Closure(
                 "n",
                  If0(/*…fact…*/),
                  e + ("fact" -> Closure(
                                 "n",
                                  If0(/*…fact…*/),
                                  e ))))
```

For initial call (`fact 3`) the body of `fact` will be evaluated in this env. extended with `[n → 3]`

One recursive call (`fact 2`) **OK.**

Environment after the **first** recursive call, i.e., for evaluating (`fact 1`).

→ Second recursive call fails

Environments for Recursion

To perform the **second** recursive call we need

$\rho_{\texttt{letrec,body}}(e) =$

```
e + ("fact" ->
      Closure(
       "n",
       If0( /*…fact…*/ ),
       e + ("fact" ->
             Closure(
             "n",
              If0( /*…fact…*/ ),
              e + ("fact" ->
                    Closure(
                    "n",
                     If0( /*…fact…*/ ),
                     e ))))))
```

Environments for Recursion

- We recognize a reoccurring pattern ...
- We need environments that never “run out” of **`fact`** definitions.
- The environment over which to close the closure should be the environment that binds the closure to a name ... i.e., the result of binding the closure
- We are running in circles here …

Environments for Recursion

- Let‘s abstract over details and make the environment to be put in the closure for a moment a parameter *E* to the environment transformer.

- Let us assume a helper function, `ρ′`, which consumes some ambient environment *e* and *E*

```
ρ′(e) =
        E => e + ("fact" ->
                            Closure(
                             "n",
                             If0( /*…fact…*/ ),
                             E
                             )
                  )
```

Environments for Recursion

- For some specific ambient environment e_0 let's set

$$\rho' e_0 = \rho'(e_0)$$

- $\rho' e_0$ consumes an E and returns an environment that extends e_0 such that recursive calls in the body of `letrec` can unfold "forever" …

Which concrete argument E_0 would enable this?

Environments for Recursion

$E_0 = \boldsymbol{\rho}'_{e_0}(E_0)$

We have to supply the answer that we want to produce!

An input value, for which the function result is the value itself, is called a **fixed-point** of the function.

It is not trivial to define a fixed point mathematically.

Is the problem easier to solve with programming?

Cyclic Environments for Recursion

Thinking algorithmically, the solution is trivial:
The environment must be a cyclic data structure!

```
ρ_letrec,body(e) = e + ("fact" ->
                        Closure("n",
                          If0( /*…fact…*/ ),
                        )
```

„The environment over which the closure closes should be the one that binds the closure..."

Outline

- **Implementing First-Class Functions**

- **Recursion**
  - Understanding recursion
  - Implementing recursion

- **Reflecting on representational choices**
  - Data Structures for environments, numbers, functions
  - Types of Interpreters

Implementing Recursion

Recall the interpreter for `let`:

```
def interp(expr: Expr, env: Env = Map()): Val = expr match {
 ...
 case Let(boundId, namedExpr, boundBody) =>
       interp(boundBody, env + (boundId -> interp(namedExpr, env)))
 ...
}
```

Implementing Recursion

An analogous interpreter for `letrec` …

```
def interp(expr: Expr, env: Env = Map()): Val = expr match {
 ...
 case LetRec(boundId, namedExpr, boundBody) =>
       interp(boundBody, env + (boundId -> interp(namedExpr, env)))
 ...
}
```

The closure will be closed over an environment that does not contain the recursive definition

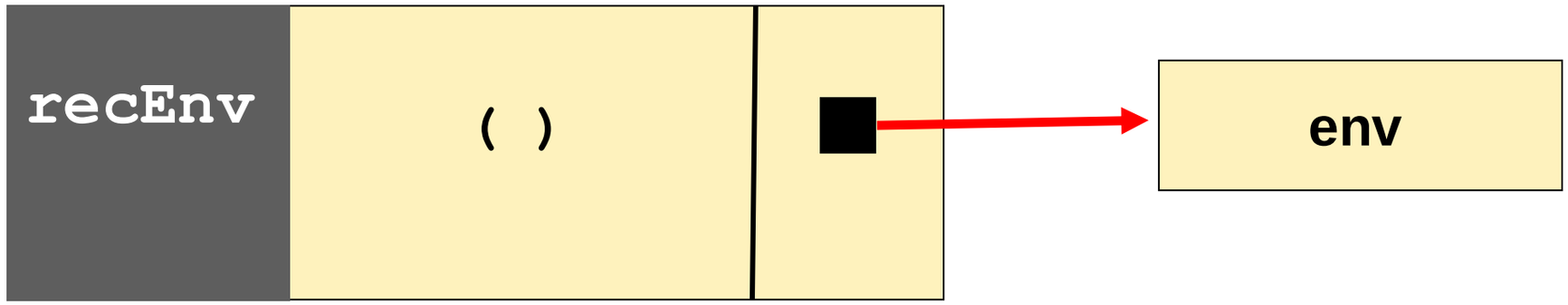
Implementing Recursion

Need to create an environment **e** that binds **boundId** to a closure that closes the function definition in **namedExpr** over **e**.

e is cyclic, since it refers to a value that, in turn, refers to **e**.

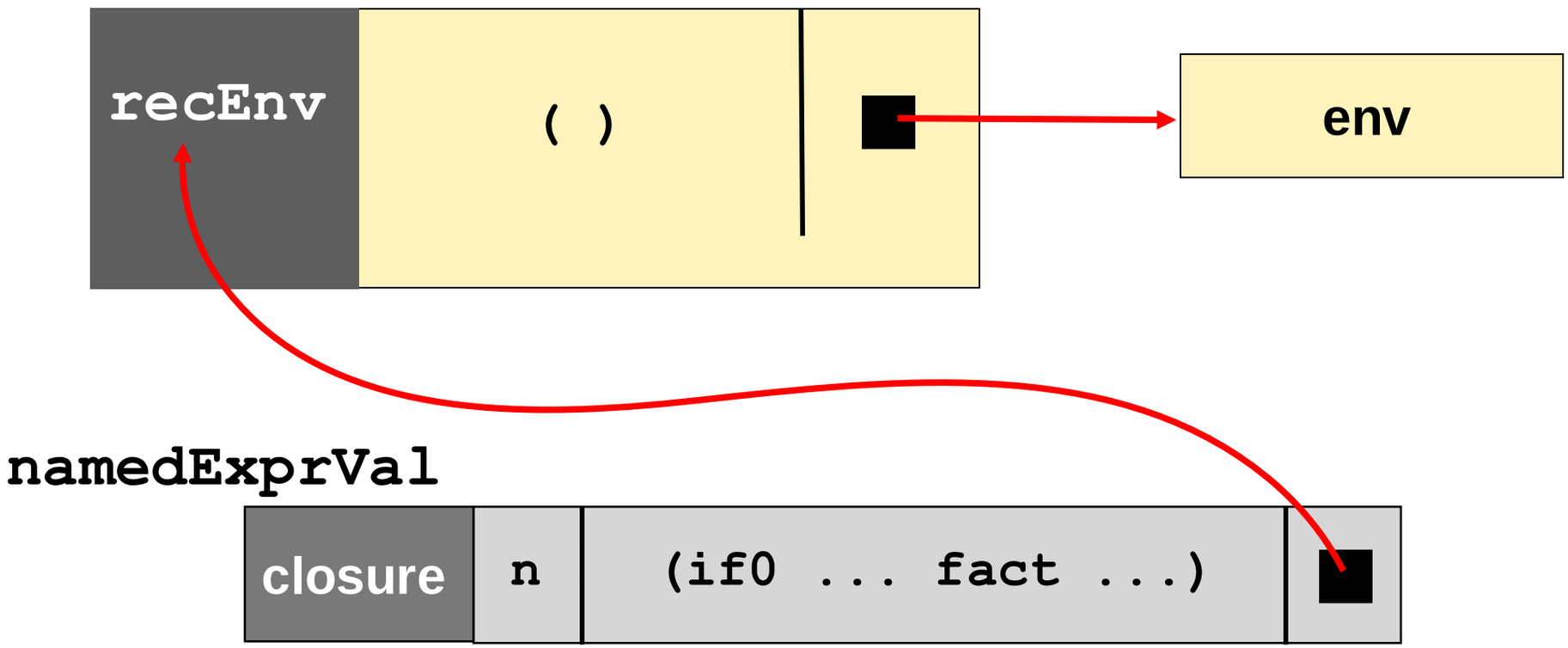
```
def interp(expr: Expr, env: Env = Map()): Val = expr match {
…
case LetRec(boundId, namedExpr, boundBody) =>
  interp(boundBody,
         recBind(boundId, namedExpr, env) )
 ...
}
```

Cyclic Environments: The Idea

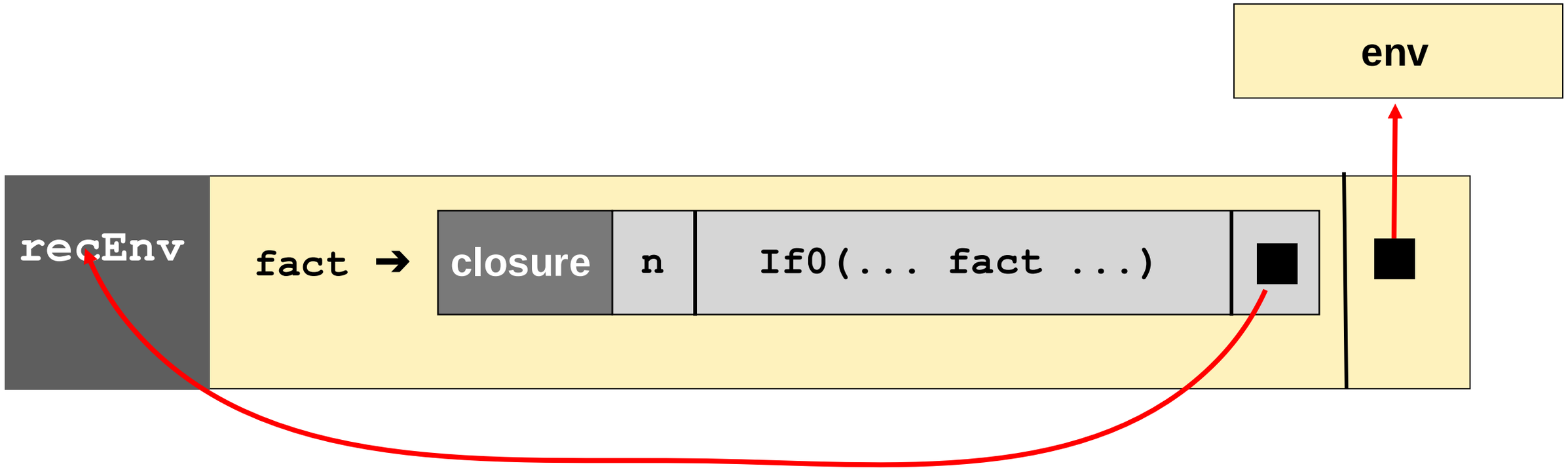


- First, create an empty mutable map and append **`env`** to it.
- Now we can interpret **`namedExpr`** in the new environment (**`recEnv`**).

Cyclic Environments: The Idea



Cyclic Environments: The Idea



Add a binding of **`fact`** to **`namedExprVal`** to **`recEnv`**

Quiz

How would you extend a CFLAE interpreter that employs substitution to support recursion?

Recursion with Substitution

```
interp(expr: RCFLAE): Value = expr match {
  ...
  case Let(boundId, namedExpr, boundBody) =>
       interp(subst(boundBody, boundId, interp(namedExpr)))

  case LetRec(boundId, namedExpr, boundBody) => ???
```

```
interp(
 (letrec fact
      (fun (n) (if0 n) (1) (* n (fact (- n 1))))
      (fact 5)))
```

What would happen, if we used the same implementation as for **`Let`**?

Interpreting **`(fact 5)`** will lead to unbound id error, since interpreting named-expr **`(fun n …)`** returns a fun-expr with a free variable in the body (**`fact`**).

Any idea?

Recursion with Substitution

```
interp(expr: RCFLAE): Value = expr match {
  ...
  case Let(boundId, namedExpr, boundBody) =>
       interp(subst(boundBody, boundId, interp(namedExpr)))

  case LetRec(boundId, namedExpr, boundBody) => ???
```

For **`LetRec`** instead of interpreting **`namedExpr`** we rather want to interpret a version of it, in which **`fact`** in the body of the function definition **`(if (x = 0) 1 else fact ...)`** is replaced with an expression that gives us as many definitions for **`fact`** as needed.

```
newNamedExpr = subst (namedExpr boundId ??? )
interp(... interp(newNamedExpr))
```

Recursion with Substitution

```
interp(
  (letrec fact
        (fun (n) (if0 n) (1) (* n (fact (- n 1)) ))
        (fact 5)))
```

What about this?

letrec fact (fun (n) (... fact ...)) (id(fact)))

```
interp(expr: RCFLAE): Value = expr match {
  ...
  case LetRec(boundId, namedExpr, boundBody) =>
    val recExpr        = LetRec(boundId, namedExpr, Id(boundId))
    val newNamedExpr   = subst(namedExpr, boundId, recExpr)
    interp(subst(boundBody, boundId, interp(newNamedExpr)))
  ...
```

Recursion with Substitution

```
interp(expr: RCFLAE): Value = expr match {
  ...
  case Let(boundId, namedExpr, boundBody) =>
       interp(subst(boundBody, boundId, interp(namedExpr)))

  case LetRec(boundId, namedExpr, boundBody) =>
    val recExpr        = LetRec(boundId, namedExpr, Id(boundId))
    val newNamedExpr   = subst(namedExpr, boundId, recExpr)
    interp(subst(boundBody, boundId, interp(newNamedExpr)))
  ...
```

Outline

- **Implementing First-Class Functions**

- **Recursion**
  - Understanding recursion
  - Implementing recursion

- **Reflecting on representational choices**
  - Data structures for environments, numbers, functions
  - Types of interpreters

Representation Choices

We made several choices about the representation of certain language concepts in our interpreter ...

- **Numbers** are represented by Scala numbers.
- **Environments** are represented by Scala maps.
- **Functions** are represented by a custom data structure (`Closure`).

Alternative Representation Choices

Next ...

- we‘ll consider what other alternatives we had ...
- we’ll reflect on the choices we made or could have made…

Alternative Representation Choices

- **Numbers** are represented by Scala numbers.

Numbers are not interesting; will keep the representation by Scala numbers

- **Environments** are represented by hash maps.
- **Functions** are represented by a custom data structure (`Closure`).

Alternative Representation for Environments

Alternative Representation of Environments

What could the alternative be?

- An environment maps identifiers to values ... it's just a (partial) function.
- Scala functions can represent environments …

```
type Env = (String => Val)

def createEnv(name: String, value: Val, oldEnv: Env): Env = ...
```

Environments as Scala Functions

```
def createEnv(name: String, value: Val, oldEnv: Env): Env =
  id => if (id == name) value
        else oldEnv(id)
```

What's lookup now?

```
def emptyEnv(name: String): Val =
    sys.error("No binding for " + name)
```

The resulting interpreter
Recursion.Meta.Environments

Environments as Scala Functions

How do we implement recursion with environments represented as Scala functions?

```
case LetRec(boundId, namedExpr, boundBody) =>
    interp(boundBody(recBind(boundId, namedExpr, env)))
```

Environments as Scala Functions

```
...
case LetRec(boundId, namedExpr, boundBody) =>
    interp(boundBody(recBind(boundId, namedExpr, env)))
```

```
def recBind(boundId: String, namedExpr: Expr, env: Env): Env = {
  ...

  (id: String) => { ... }

  ...
}
```

How would **`recBind`** look like?

We know, it will return a function. But what's in the wholes?

Environments as Scala Functions

```
def recBind(boundId: Symbol, namedExpr: Expr, env: Env): Env =
  def recEnv: Env =
     (id: String) =>
      if id == boundId
        then interp(namedExpr, recEnv)
     else env(id)
 recEnv
```

Alternative Representation for Functions

Alternative Representation of Functions

Can you think of an alternative to the custom data structure for representing functions?

We could use a Scala function...

```
sealed abstract class Val
case class Num(n: Int) extends Val
case class Closure(f: Val => Val) extends Val
```

How would the interpreter change?

Representing Functions as Scala Functions

```
def interp(expr: Expr, env: Env = emptyEnv): Val = expr match {
  . . .

 case Fun(arg, body) =>
  Closure( argValue => interp(body, createEnv(arg, argValue, env)) )
 . . .

  case App(funExpr, argExpr) =>
   val funV = interp(funExpr, env)
   val argV = interp(argExpr, env)
   funV match {
    case Closure(f) => f(argV)
    case _ => sys.error("can only apply functions, but got: " + funV)
   }
}
```

Outline

- **Implementing First-Class Functions**

- **Recursion**
  - Understanding recursion
  - Implementing recursion

- **Reflecting on representational choices**
  - Data structures for environments, numbers, functions
  - Types of interpreters

Types of Interpreters

- **Syntactic interpreter**
  - uses the interpreting language only for the purpose of representing terms of the interpreted language,
  - implements all the corresponding behavior explicitly
- **Meta interpreter**
  - uses features of the interpreting language to directly implement behavior of the interpreted language
- **Meta-circular interpreter**
  - a meta interpreter in which the interpreting and interpreted language are the same

Types of Interpreters

How do the interpreters that we have seen so far fit into these definitions?

Environments as Maps versus Functions

- The original RCFLAE is a syntactic interpreter

- The interpreter for RCFLAE that uses functional environments is a meta-interpreter. It uses Scala's recursion to implement recursion

```
def recBind(boundId: Symbol, namedExpr: Expr, env: Env): Env =
  def recEnv: Env =
     (id: String) =>
      if (id == boundId) interp(namedExpr, recEnv)
      else env(id)
  recEnv
```

How do you (dis)like the interpreter that uses functional environments?

Environments as Maps versus Functions

- We set out to add recursion to RCFAE and reduced this to the problem of defining cyclic environments.

- But then we implemented cyclic environments by falling right back on Scala's recursive functions!

Environments as Maps versus Functions

- The syntactic interpreter is superior: it doesn't rely on advanced knowledge of Scala and its advanced features

- Power and peril of meta-interpretive choices
  - Power: Easy to add recursion. If the goal is to add it quickly, while minimizing error, we can exploit the effort put into implementing recursion for Scala.
  - Peril: the implementation does not hold descriptive power: still begs the question of how to implement recursion from scratch.

Functions as Structures versus as Functions

Our new function representation might be a bit confusing

There are questions that are not obvious, when we rely on Scala's functions …

Functions as Scala Functions

Is the body of a function evaluated at the definition site, i.e., in the **`Fun`** branch of the interpreter?

```
def interp(expr: Expr, env: Env = emptyEnv): Val = expr match {
  . . .

 case Fun(arg, body) =>
  Closure( argValue => interp(body, createEnv(arg, argValue, env)) )
 . . .

 case App(funExpr, argExpr) =>
  val funV = interp(funExpr, env)
  val argV = interp(argExpr, env)
  funV match {
   case Closure(f) => f(argV)
   case _ => sys.error("can only apply functions, but got: " + funV)
  }
}
```

No, because the body is "under a lambda"

Functions as Scala Functions

Which environment is used for function application?

Do we implement static scoping or dynamic scoping?

```
def interp(expr: Expr, env: Env = emptyEnv): Val = expr match {
  . . .

 case Fun(arg, body) =>
  Closure( argValue => interp(body, createEnv(arg, argValue, env)) )
 . . .

 case App(funExpr, argExpr) =>
  val funV = interp(funExpr, env)
  val argV = interp(argExpr, env)
  funV match {
   case Closure(f) => f(argV)
   case _ => sys.error("can only apply functions, but got: " + funV)
  }
}
```

Static scoping, because Scala is statically scoped

Pros and Cons of Meta Interpreters

- If the interpreted and the interpreter language match closely, a meta interpreter can be very easy to write

- If they do not match it can be hard
  - Static scoping vs dynamic scoping
  - Lazy vs strict evaluation
  - …

- A meta interpreter does not help in understanding the features we are implementing
  - Started implementing recursion based on a cyclic environments, which are arguably easier to understand than recursion and than ended up implementing cyclic environments in terms of Scala's recursion!

When to Use Meta Interpretation

- Use a meta interpreter approach for features we understand (e.g., numbers).
- But write a syntactic interpreter for features under examination.
- Once we understand features, we can replace them with a meta interpreter to move on to more complex features.